

Q1. Smart University Payroll Management System

Score: 25.00 TC: 3/3 passed

CODE

```
import java.util.*;

class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = Integer.parseInt(nextNonEmptyLine(sc));

        Employee[] emp = new Employee[n];

        for (int i = 0; i < n; i++) {
            String type = nextNonEmptyLine(sc);

            int id = Integer.parseInt(nextNonEmptyLine(sc));
            String name = nextNonEmptyLine(sc);
            String dept = nextNonEmptyLine(sc);
            double basicSalary = Double.parseDouble(nextNonEmptyLine(sc));

            if (type.equalsIgnoreCase("TeachingFaculty")) {
                int teachingHours = Integer.parseInt(nextNonEmptyLine(sc));
                double incentivePerHour = Double.parseDouble(nextNonEmptyLine(sc));
                emp[i] = new TeachingFaculty(id, name, dept, basicSalary, teachingHours, incentivePerHour);
            } else if (type.equalsIgnoreCase("TechnicalStaff")) {
                int overtimeHours = Integer.parseInt(nextNonEmptyLine(sc));
                double overtimeRate = Double.parseDouble(nextNonEmptyLine(sc));
                emp[i] = new TechnicalStaff(id, name, dept, basicSalary, overtimeHours, overtimeRate);
            } else if (type.equalsIgnoreCase("VisitingFaculty")) {
                int lecturesHandled = Integer.parseInt(nextNonEmptyLine(sc));
                double paymentPerLecture = Double.parseDouble(nextNonEmptyLine(sc));
                emp[i] = new VisitingFaculty(id, name, dept, basicSalary, lecturesHandled, paymentPerLecture);
            }
        }

        String searchLine = nextNonEmptyLine(sc);
        int searchId = Integer.parseInt(nextNonEmptyLine(sc));

        System.out.println("Employee Details");
        System.out.println();

        double total = 0.0;
        double maxSalary = Double.NEGATIVE_INFINITY;
        Employee highest = null;

        for (Employee e : emp) {
            e.display();
            double salary = e.calculateSalary();
            System.out.println("Monthly Salary : " + salary);
            System.out.println();

            total += salary;
            if (salary > maxSalary) {
                maxSalary = salary;
                highest = e;
            }
        }

        System.out.println("Highest Paid Employee");
        System.out.println("Employee ID : " + highest.getEmployeeId());
        System.out.println("Salary : " + highest.calculateSalary());
        System.out.println();
    }
}
```

```
        System.out.println("Average Salary");
        System.out.printf("%.2f\n", total / n);
        System.out.println();

        Employee foundEmp = null;
        for (Employee e : emp) {
            if (e.getEmployeeId() == searchId) {
                foundEmp = e;
                break;
            }
        }

        if (foundEmp != null) {
            System.out.println("Employee Found");
            System.out.println("Employee ID : " + foundEmp.getEmployeeId());
            System.out.println("Name : " + foundEmp.getEmployeeName());
            System.out.println("Salary : " + foundEmp.calculateSalary());
        } else {
            System.out.println("Employee Not Found");
        }
    }

    private static String nextNonEmptyLine(Scanner sc) {
        while (sc.hasNextLine()) {
            String line = sc.nextLine().trim();
            if (!line.isEmpty()) return line;
        }
    }
}
```

Q1. Smart University Payroll Management System

Score: 25.00 TC: 3/3 passed

CODE

```
import java.util.*;

class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = Integer.parseInt(nextNonEmptyLine(sc));

        Employee[] emp = new Employee[n];

        for (int i = 0; i < n; i++) {
            String type = nextNonEmptyLine(sc);

            int id = Integer.parseInt(nextNonEmptyLine(sc));
            String name = nextNonEmptyLine(sc);
            String dept = nextNonEmptyLine(sc);
            double basicSalary = Double.parseDouble(nextNonEmptyLine(sc));

            if (type.equalsIgnoreCase("TeachingFaculty")) {
                int teachingHours = Integer.parseInt(nextNonEmptyLine(sc));
                double incentivePerHour = Double.parseDouble(nextNonEmptyLine(sc));
                emp[i] = new TeachingFaculty(id, name, dept, basicSalary, teachingHours, incentivePerHour);
            } else if (type.equalsIgnoreCase("TechnicalStaff")) {
                int overtimeHours = Integer.parseInt(nextNonEmptyLine(sc));
                double overtimeRate = Double.parseDouble(nextNonEmptyLine(sc));
                emp[i] = new TechnicalStaff(id, name, dept, basicSalary, overtimeHours, overtimeRate);
            } else if (type.equalsIgnoreCase("VisitingFaculty")) {
                int lecturesHandled = Integer.parseInt(nextNonEmptyLine(sc));
                double paymentPerLecture = Double.parseDouble(nextNonEmptyLine(sc));
                emp[i] = new VisitingFaculty(id, name, dept, basicSalary, lecturesHandled, paymentPerLecture);
            }
        }

        String searchLine = nextNonEmptyLine(sc);
        int searchId = Integer.parseInt(nextNonEmptyLine(sc));

        System.out.println("Employee Details");
        System.out.println();

        double total = 0.0;
        double maxSalary = Double.NEGATIVE_INFINITY;
        Employee highest = null;

        for (Employee e : emp) {
            e.display();
            double salary = e.calculateSalary();
            System.out.println("Monthly Salary : " + salary);
            System.out.println();

            total += salary;
            if (salary > maxSalary) {
                maxSalary = salary;
                highest = e;
            }
        }

        System.out.println("Highest Paid Employee");
        System.out.println("Employee ID : " + highest.getEmployeeId());
        System.out.println("Salary : " + highest.calculateSalary());
        System.out.println();
    }
}
```

```

double total = 0.0;
double maxSalary = Double.NEGATIVE_INFINITY;
Employee highest = null;

for (Employee e : emp) {
    e.display();
    double salary = e.calculateSalary();
    System.out.println("Monthly Salary : " + salary);
    System.out.println();

    total += salary;
    if (salary > maxSalary) {
        maxSalary = salary;
        highest = e;
    }
}

System.out.println("Highest Paid Employee");
System.out.println("Employee ID : " + highest.getEmployeeId());
System.out.println("Salary : " + highest.calculateSalary());
System.out.println();

```

SIMATS Engineering • Department of CSE • CSA0925 — Programming In java • 14-08-2026 • Category: CO1-AT3 - CODING CHALLENGE • Student Submissions Report

```

System.out.println("Average Salary");
System.out.printf("%.2f\n", total / n);
System.out.println();

Employee foundEmp = null;
for (Employee e : emp) {
    if (e.getEmployeeId() == searchId) {
        foundEmp = e;
        break;
    }
}

if (foundEmp != null) {
    System.out.println("Employee Found");
    System.out.println("Employee ID : " + foundEmp.getEmployeeId());
    System.out.println("Name : " + foundEmp.getEmployeeName());
    System.out.println("Salary : " + foundEmp.calculateSalary());
} else {
    System.out.println("Employee Not Found");
}
}

private static String nextNonEmptyLine(Scanner sc) {
    while (sc.hasNextLine()) {
        String line = sc.nextLine().trim();
        if (!line.isEmpty()) return line;
    }
    return "";
}

}

class Employee {
    private int employeeId;
    private String employeeName;
    private String department;
    private double basicSalary;

    public Employee() {
    }

    public Employee(int employeeId, String employeeName, String department, double basicSalary) {
        this.employeeId = employeeId;
        this.employeeName = employeeName;
        this.department = department;
        this.basicSalary = basicSalary;
    }

    public int getEmployeeId() {
        return employeeId;
    }

    public void setEmployeeId(int employeeId) {
        this.employeeId = employeeId;
    }

    public String getEmployeeName() {
        return employeeName;
    }

    public void setEmployeeName(String employeeName) {
        this.employeeName = employeeName;
    }
}

```

SIMATS Engineering • Department of CSE • CSA0925 — Programming In java • 14-08-2026 • Category: CO1-AT3 - CODING CHALLENGE • Student Submissions Report

```

}

    public String getDepartment() {
        return department;
    }

    public void setDepartment(String department) {
        this.department = department;
    }

    public double getBasicSalary() {
        return basicSalary;
    }

    public void setBasicSalary(double basicSalary) {
        this.basicSalary = basicSalary;
    }
}

```

```
        System.out.println("Average Salary");
        System.out.printf("%.2f%n", total / n);
        System.out.println();

        Employee foundEmp = null;
        for (Employee e : emp) {
            if (e.getEmployeeId() == searchId) {
                foundEmp = e;
                break;
            }
        }

        if (foundEmp != null) {
            System.out.println("Employee Found");
            System.out.println("Employee ID : " + foundEmp.getEmployeeId());
            System.out.println("Name : " + foundEmp.getEmployeeName());
            System.out.println("Salary : " + foundEmp.calculateSalary());
        } else {
            System.out.println("Employee Not Found");
        }
    }

    private static String nextNonEmptyLine(Scanner sc) {
        while (sc.hasNextLine()) {
            String line = sc.nextLine().trim();
            if (!line.isEmpty()) return line;
        }
        return "";
    }
}

class Employee {
    private int employeeId;
    private String employeeName;
    private String department;
    private double basicSalary;

    public Employee() {
    }

    public Employee(int employeeId, String employeeName, String department, double basicSalary) {
        this.employeeId = employeeId;
        this.employeeName = employeeName;
        this.department = department;
        this.basicSalary = basicSalary;
    }

    public int getEmployeeId() {
        return employeeId;
    }

    public void setEmployeeId(int employeeId) {
        this.employeeId = employeeId;
    }

    public String getEmployeeName() {
        return employeeName;
    }

    public void setEmployeeName(String employeeName) {
        this.employeeName = employeeName;
    }
}
```

```

public Employee(int employeeId, String employeeName, String department, double basicSalary) {
    this.employeeId = employeeId;
    this.employeeName = employeeName;
    this.department = department;
    this.basicSalary = basicSalary;
}

public int getEmployeeId() {
    return employeeId;
}

public void setEmployeeId(int employeeId) {
    this.employeeId = employeeId;
}

public String getEmployeeName() {
    return employeeName;
}

public void setEmployeeName(String employeeName) {
    this.employeeName = employeeName;
}

```

SIMATS Engineering • Department of CSE • CSA0925 — Programming In java • 14-08-2026 • Category: CO1-AT3 - CODING CHALLENGE • Student Submissions Report

```

}

public String getDepartment() {
    return department;
}

public void setDepartment(String department) {
    this.department = department;
}

public double getBasicSalary() {
    return basicSalary;
}

public void setBasicSalary(double basicSalary) {
    this.basicSalary = basicSalary;
}

public void display() {
    System.out.println("Employee ID : " + employeeId);
    System.out.println("Name : " + employeeName);
    System.out.println("Department : " + department);
}

public double calculateSalary() {
    return basicSalary;
}
}

class TeachingFaculty extends Employee {
    private int teachingHours;
    private double incentivePerHour;

    public TeachingFaculty(int employeeId, String employeeName, String department,
        double basicSalary, int teachingHours, double incentivePerHour) {
        super(employeeId, employeeName, department, basicSalary);
        this.teachingHours = teachingHours;
        this.incentivePerHour = incentivePerHour;
    }

    @Override
    public double calculateSalary() {
        return getBasicSalary() + (teachingHours * incentivePerHour);
    }
}

class TechnicalStaff extends Employee {
    private int overtimeHours;
    private double overtimeRate;

    public TechnicalStaff(int employeeId, String employeeName, String department,
        double basicSalary, int overtimeHours, double overtimeRate) {
        super(employeeId, employeeName, department, basicSalary);
        this.overtimeHours = overtimeHours;
        this.overtimeRate = overtimeRate;
    }

    @Override
    public double calculateSalary() {
        return getBasicSalary() + (overtimeHours * overtimeRate);
    }
}
}

```

SIMATS Engineering • Department of CSE • CSA0925 — Programming In java • 14-08-2026 • Category: CO1-AT3 - CODING CHALLENGE • Student Submissions Report

```

class VisitingFaculty extends Employee {
    private int lecturesHandled;
    private double paymentPerLecture;

    public VisitingFaculty(int employeeId, String employeeName, String department,
        double basicSalary, int lecturesHandled, double paymentPerLecture) {
        super(employeeId, employeeName, department, basicSalary);
        this.lecturesHandled = lecturesHandled;
        this.paymentPerLecture = paymentPerLecture;
    }

    @Override
    public double calculateSalary() {
        return lecturesHandled * paymentPerLecture;
    }
}
}

```

Submissions Report

```
}

public String getDepartment() {
    return department;
}

public void setDepartment(String department) {
    this.department = department;
}

public double getBasicSalary() {
    return basicSalary;
}

public void setBasicSalary(double basicSalary) {
    this.basicSalary = basicSalary;
}

public void display() {
    System.out.println("Employee ID : " + employeeId);
    System.out.println("Name : " + employeeName);
    System.out.println("Department : " + department);
}

public double calculateSalary() {
    return basicSalary;
}
}

class TeachingFaculty extends Employee {
    private int teachingHours;
    private double incentivePerHour;

    public TeachingFaculty(int employeeId, String employeeName, String department,
                           double basicSalary, int teachingHours, double incentivePerHour) {
        super(employeeId, employeeName, department, basicSalary);
        this.teachingHours = teachingHours;
        this.incentivePerHour = incentivePerHour;
    }

    @Override
    public double calculateSalary() {
        return getBasicSalary() + (teachingHours * incentivePerHour);
    }
}

class TechnicalStaff extends Employee {
    private int overtimeHours;
    private double overtimeRate;

    public TechnicalStaff(int employeeId, String employeeName, String department,
                          double basicSalary, int overtimeHours, double overtimeRate) {
        super(employeeId, employeeName, department, basicSalary);
        this.overtimeHours = overtimeHours;
        this.overtimeRate = overtimeRate;
    }

    @Override
    public double calculateSalary() {
        return getBasicSalary() + (overtimeHours * overtimeRate);
    }
}
```



ba18a565-b2d0-40ad-a8af-81...



```
@Override
public double calculateSalary() {
    return getBasicSalary() + (overtimeHours * overtimeRate);
}
}
```

SIMATS Engineering • Department of CSE • CSA0925 — Programming In java • 14-08-2026 • Category: CO1-AT3 - CODING CHALLENGE • Student Submissions Report

```
class VisitingFaculty extends Employee {
    private int lecturesHandled;
    private double paymentPerLecture;

    public VisitingFaculty(int employeeId, String employeeName, String department,
                           double basicSalary, int lecturesHandled, double paymentPerLecture) {
        super(employeeId, employeeName, department, basicSalary);
        this.lecturesHandled = lecturesHandled;
        this.paymentPerLecture = paymentPerLecture;
    }

    @Override
    public double calculateSalary() {
        return lecturesHandled * paymentPerLecture;
    }
}
```

INPUT

```
3
TeachingFaculty
101
Dr.Arun
CSE
50000
20
500

TechnicalStaff
102
Mr.Ravi
ECE
35000
15
300

VisitingFaculty
103
Mr.Kumar
IT
0
40
1200

Search Employee ID:
102
```

OUTPUT

```
Employee Details

Employee ID : 101
Name : Dr.Arun
Department : CSE
Monthly Salary : 60000.0

Employee ID : 102
Name : Mr.Ravi
Department : ECE
Monthly Salary : 39500.0

Employee ID : 103
Name : Mr.Kumar
Department : IT
Monthly Salary : 48000.0

Highest Paid Employee
Employee ID : 101
Salary : 60000.0

Average Salary
49166.67

Employee Found
Employee ID : 102
Name : Mr.Ravi
Salary : 39500.0
```

SIMATS Engineering • Department of CSE • CSA0925 — Programming In java • 14-08-2026 • Category: CO1-AT3 - CODING CHALLENGE • Student Submissions Report

Q2. Smart Vehicle Insurance Claim System

Score: 25.00 TC: 3/3 passed

CODE

```
import java.util.*;

public class Main {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
    }
}
```



Rotate screen



Play



Share



Search

```
class VisitingFaculty extends Employee {
    private int lecturesHandled;
    private double paymentPerLecture;

    public VisitingFaculty(int employeeId, String employeeName, String department,
                           double basicSalary, int lecturesHandled, double paymentPerLecture) {
        super(employeeId, employeeName, department, basicSalary);
        this.lecturesHandled = lecturesHandled;
        this.paymentPerLecture = paymentPerLecture;
    }

    @Override
    public double calculateSalary() {
        return lecturesHandled * paymentPerLecture;
    }
}
```

INPUT

```
3

TeachingFaculty
101
Dr.Arun
CSE
50000
20
500

TechnicalStaff
102
Mr.Ravi
ECE
35000
15
300

VisitingFaculty
103
Mr.Kumar
IT
0
40
1200

Search Employee ID:
102
```

OUTPUT

```
Employee Details

Employee ID : 101
Name : Dr.Arun
Department : CSE
Monthly Salary : 60000.0

Employee ID : 102
Name : Mr.Ravi
Department : ECE
Monthly Salary : 39500.0

Employee ID : 103
Name : Mr.Kumar
Department : IT
Monthly Salary : 48000.0

Highest Paid Employee
Employee ID : 101
Salary : 60000.0

Average Salary
49166.67

Employee Found
Employee ID : 102
Name : Mr.Ravi
Salary : 39500.0
```



```
Name : Mr.Kumar
Department : IT
Monthly Salary : 48000.0

Highest Paid Employee
Employee ID : 101
Salary : 60000.0

Average Salary
49166.67

Employee Found
Employee ID : 102
Name : Mr.Ravi
Salary : 39500.0
```

SIMATS Engineering • Department of CSE • CSA0925 — Programming In java • 14-08-2026 • Category: CO1-AT3 - CODING CHALLENGE • Student Submissions Report

Q2. Smart Vehicle Insurance Claim System

Score: 25.00 TC: 3/3 passed

CODE

```
import java.util.*;

public class Main {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = readNextInt(sc);
        Vehicle[] vehicles = new Vehicle[n];

        for (int i = 0; i < n; i++) {
            String type = readNextNonEmpty(sc);    // Car/Bike/Truck
            String vehicleNumber = readNextNonEmpty(sc);
            String ownerName = readNextNonEmpty(sc);
            double insuredAmount = readNextDouble(sc);

            if (type.equalsIgnoreCase("Car")) {
                vehicles[i] = new Car(vehicleNumber, ownerName, insuredAmount);
            } else if (type.equalsIgnoreCase("Bike")) {
                vehicles[i] = new Bike(vehicleNumber, ownerName, insuredAmount);
            } else {
                vehicles[i] = new Truck(vehicleNumber, ownerName, insuredAmount);
            }
        }

        String searchVehicle = readSearchVehicle(sc);

        // Compute and print all vehicles
        Vehicle highest = null;
        double highestClaim = Double.NEGATIVE_INFINITY;

        for (int i = 0; i < n; i++) {
            Vehicle v = vehicles[i];
            double claim = v.calculateClaimAmount();

            System.out.println("Vehicle : " + v.getVehicleNumber());
            System.out.println("Claim Amount : " + claim);
            if (i != n - 1) System.out.println();

            if (claim > highestClaim) {
                highestClaim = claim;
                highest = v;
            }
        }

        // Highest claim
        System.out.println();
        System.out.println("Highest Claim Vehicle");
        System.out.println(highest.getVehicleNumber());
        System.out.println(highestClaim);

        // Search
        boolean found = false;
        System.out.println();
        if (searchVehicle != null && !searchVehicle.isEmpty()) {
            for (Vehicle v : vehicles) {
                if (v.getVehicleNumber().equalsIgnoreCase(searchVehicle)) {
                    System.out.println("Vehicle Found");
                    System.out.println("Owner : " + v.getOwnerName());
                    System.out.println("Claim Amount : " + v.calculateClaimAmount());
                }
            }
        }
    }
}
```

SIMATS Engineering • Department of CSE • CSA0925 — Programming In java • 14-08-2026 • Category: CO1-AT3 - CODING CHALLENGE • Student Submissions Report

```
        found = true;
        break;
    }
}

if (!found) {
    System.out.println("Vehicle Not Found");
}

sc.close();
}

// Reads "Search Vehicle: <num>" robustly (handles tokens with/without space/newlines)
private static String readSearchVehicle(Scanner sc) {
    while (sc.hasNext()) {
        String token = sc.next().trim();
        if (token.isEmpty()) continue;

        // token could be "Search" then next tokens include "Vehicle:" and number
        if (token.equalsIgnoreCase("Search")) {
            // keep reading and find a number after Vehicle:
            while (sc.hasNext()) {
                String token2 = sc.next().trim();
                if (token2.isEmpty()) continue;
                if (token2.startsWith("Vehicle:")) {
                    continue;
                }
                return token2;
            }
        }
    }
    return null;
}
```

Q2. Smart Vehicle Insurance Claim System

Score: 25.00 TC: 3/3 passed

CODE

```
import java.util.*;

public class Main {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = readNextInt(sc);
        Vehicle[] vehicles = new Vehicle[n];

        for (int i = 0; i < n; i++) {
            String type = readNextNonEmpty(sc);    // Car/Bike/Truck
            String vehicleNumber = readNextNonEmpty(sc);
            String ownerName = readNextNonEmpty(sc);
            double insuredAmount = readNextDouble(sc);

            if (type.equalsIgnoreCase("Car")) {
                vehicles[i] = new Car(vehicleNumber, ownerName, insuredAmount);
            } else if (type.equalsIgnoreCase("Bike")) {
                vehicles[i] = new Bike(vehicleNumber, ownerName, insuredAmount);
            } else {
                vehicles[i] = new Truck(vehicleNumber, ownerName, insuredAmount);
            }
        }

        String searchVehicle = readSearchVehicle(sc);

        // Compute and print all vehicles
        Vehicle highest = null;
        double highestClaim = Double.NEGATIVE_INFINITY;

        for (int i = 0; i < n; i++) {
            Vehicle v = vehicles[i];
            double claim = v.calculateClaimAmount();

            System.out.println("Vehicle : " + v.getVehicleNumber());
            System.out.println("Claim Amount : " + claim);
            if (i != n - 1) System.out.println();

            if (claim > highestClaim) {
                highestClaim = claim;
                highest = v;
            }
        }

        // Highest claim
        System.out.println();
        System.out.println("Highest Claim Vehicle");
        System.out.println(highest.getVehicleNumber());
        System.out.println(highestClaim);

        // Search
        boolean found = false;
        System.out.println();
        if (searchVehicle != null && !searchVehicle.isEmpty()) {
            for (Vehicle v : vehicles) {
                if (v.getVehicleNumber().equalsIgnoreCase(searchVehicle)) {
                    System.out.println("Vehicle Found");
                    System.out.println("Owner : " + v.getOwnerName());
                    System.out.println("Claim Amount : " + v.calculateClaimAmount());
                }
            }
        }
    }
}
```



ba18a565-b2d0-40ad-a8af-81...



```

System.out.println(highestClaim);

// Search
boolean found = false;
System.out.println();
if (searchVehicle != null && !searchVehicle.isEmpty()) {
    for (Vehicle v : vehicles) {
        if (v.getVehicleNumber().equalsIgnoreCase(searchVehicle)) {
            System.out.println("Vehicle Found");
            System.out.println("Owner : " + v.getOwnerName());
            System.out.println("Claim Amount : " + v.calculateClaimAmount());
        }
    }
}

```

SIMATS Engineering • Department of CSE • CSA0925 — Programming In java • 14-08-2026 • Category: CO1-AT3 - CODING CHALLENGE • Student Submissions Report

```

        found = true;
        break;
    }
}

if (!found) {
    System.out.println("Vehicle Not Found");
}

sc.close();
}

// Reads "Search Vehicle: <num>" robustly (handles tokens with/without space/newlines)
private static String readSearchVehicle(Scanner sc) {
    while (sc.hasNext()) {
        String token = sc.next().trim();
        if (token.isEmpty()) continue;

        // token could be "Search" then next tokens include "Vehicle:" and number
        if (token.equalsIgnoreCase("Search")) {
            // keep reading and find a number after Vehicle:
            while (sc.hasNext()) {
                String t = sc.next().trim();
                if (t.isEmpty()) continue;

                if (t.equalsIgnoreCase("Vehicle") || t.equalsIgnoreCase("Vehicle:")) {
                    return readNextNonEmpty(sc);
                }
                if (t.toLowerCase().startsWith("vehicle:")) {
                    String[] parts = t.split(":", 2);
                    if (parts.length == 2) return parts[1].trim();
                }
                // Sometimes the very next token is the number itself
                if (!t.equalsIgnoreCase("Vehicle") && !t.equalsIgnoreCase("Vehicle:") && !t.equalsIgnoreCase("Search")) {
                    // if it looks like vehicle number, return it
                    return t;
                }
            }
        }

        // token could be "SearchVehicle:TN..." or "Search:TN..." etc
        if (token.toLowerCase().startsWith("search")) {
            int idx = token.indexOf(':');
            if (idx != -1 && idx + 1 < token.length()) {
                return token.substring(idx + 1).trim();
            }
            // else read next non-empty token as potential number
            String next = readNextNonEmpty(sc);
            if (!next.isEmpty()) return next;
        }

        // token could be "Vehicle:TN..." directly
        if (token.toLowerCase().startsWith("vehicle:")) {
            String[] parts = token.split(":", 2);
            if (parts.length == 2) return parts[1].trim();
        }

        // token could be exactly "Vehicle:" then number next
        if (token.equalsIgnoreCase("Vehicle:") || token.equalsIgnoreCase("Vehicle")) {
            return readNextNonEmpty(sc);
        }
    }
}
return "";

```

SIMATS Engineering • Department of CSE • CSA0925 — Programming In java • 14-08-2026 • Category: CO1-AT3 - CODING CHALLENGE • Student Submissions Report

```

    }

    private static String readNextNonEmpty(Scanner sc) {
        while (sc.hasNext()) {
            String s = sc.next();
            if (s == null) continue;
            s = s.trim();
        }
    }

```



Rotate screen



Play



Share



Search

```
        found = true;
        break;
    }
}

if (!found) {
    System.out.println("Vehicle Not Found");
}

sc.close();
}

// Reads "Search Vehicle: <num>" robustly (handles tokens with/without space/newlines)
private static String readSearchVehicle(Scanner sc) {
    while (sc.hasNext()) {
        String token = sc.next().trim();
        if (token.isEmpty()) continue;

        // token could be "Search" then next tokens include "Vehicle:" and number
        if (token.equalsIgnoreCase("Search")) {
            // keep reading and find a number after Vehicle:
            while (sc.hasNext()) {
                String t = sc.next().trim();
                if (t.isEmpty()) continue;

                if (t.equalsIgnoreCase("Vehicle") || t.equalsIgnoreCase("Vehicle:")) {
                    return readNextNonEmpty(sc);
                }
                if (t.toLowerCase().startsWith("vehicle:")) {
                    String[] parts = t.split(":", 2);
                    if (parts.length == 2) return parts[1].trim();
                }
                // Sometimes the very next token is the number itself
                if (!t.equalsIgnoreCase("Vehicle") && !t.equalsIgnoreCase("Vehicle:") && !t.equalsIgnoreCase("Search")) {
                    // if it looks like vehicle number, return it
                    return t;
                }
            }
        }

        // token could be "SearchVehicle:TN..." or "Search:TN..." etc
        if (token.toLowerCase().startsWith("search")) {
            int idx = token.indexOf(':');
            if (idx != -1 && idx + 1 < token.length()) {
                return token.substring(idx + 1).trim();
            }
            // else read next non-empty token as potential number
            String next = readNextNonEmpty(sc);
            if (!next.isEmpty()) return next;
        }

        // token could be "Vehicle:TN..." directly
        if (token.toLowerCase().startsWith("vehicle:")) {
            String[] parts = token.split(":", 2);
            if (parts.length == 2) return parts[1].trim();
        }

        // token could be exactly "Vehicle:" then number next
        if (token.equalsIgnoreCase("Vehicle:") || token.equalsIgnoreCase("Vehicle")) {
            return readNextNonEmpty(sc);
        }
    }
    return "";
}
```

```
        public String getOwnerName() {
            return ownerName;
        }

        public double getInsuredAmount() {
            return insuredAmount;
        }

        public abstract double calculateClaimAmount();
    }

    class Car extends Vehicle {
        public Car(String vehicleNumber, String ownerName, double insuredAmount) {
            super(vehicleNumber, ownerName, insuredAmount);
        }

        @Override
        public double calculateClaimAmount() {
            return getInsuredAmount() * 0.80;
        }
    }
}
```

```
    }
}

class Bike extends Vehicle {
    public Bike(String vehicleNumber, String ownerName, double insuredAmount) {
        super(vehicleNumber, ownerName, insuredAmount);
    }

    @Override
    public double calculateClaimAmount() {
        return getInsuredAmount() * 0.60;
    }
}

class Truck extends Vehicle {
    public Truck(String vehicleNumber, String ownerName, double insuredAmount) {
        super(vehicleNumber, ownerName, insuredAmount);
    }

    @Override
    public double calculateClaimAmount() {
        return getInsuredAmount() * 0.90;
    }
}
}
```

INPUT

```
3
Car
TN10AB1234
Ramesh
800000

Bike
TN09XY5678
Suresh
100000

Truck
TN11TR9999
Logan
1500000

Search Vehicle:
TN09XY5678
```

OUTPUT

```
Vehicle : TN10AB1234
Claim Amount : 640000.0

Vehicle : TN09XY5678
Claim Amount : 60000.0

Vehicle : TN11TR9999
Claim Amount : 1350000.0

Highest Claim Vehicle
TN11TR9999
1350000.0

Vehicle Found
Owner : Suresh
Claim Amount : 60000.0
```

Student 34 **Prajaan K** 192524388RD

0 / 2 submitted

Q1. Smart University Payroll Management System

— Not Submitted —

Q2. Smart Vehicle Insurance Claim System

— Not Submitted —

Submissions Report

```
}  
}  
  
class Bike extends Vehicle {  
    public Bike(String vehicleNumber, String ownerName, double insuredAmount) {  
        super(vehicleNumber, ownerName, insuredAmount);  
    }  
  
    @Override  
    public double calculateClaimAmount() {  
        return getInsuredAmount() * 0.60;  
    }  
}  
  
class Truck extends Vehicle {  
    public Truck(String vehicleNumber, String ownerName, double insuredAmount) {  
        super(vehicleNumber, ownerName, insuredAmount);  
    }  
  
    @Override  
    public double calculateClaimAmount() {  
        return getInsuredAmount() * 0.90;  
    }  
}
```

INPUT

```
3  
  
Car  
TN10AB1234  
Ramesh  
800000  
  
Bike  
TN09XY5678  
Suresh  
100000  
  
Truck  
TN11TR9999  
Logan  
1500000  
  
Search Vehicle:  
TN09XY5678
```

OUTPUT

```
Vehicle : TN10AB1234  
Claim Amount : 640000.0  
  
Vehicle : TN09XY5678  
Claim Amount : 60000.0  
  
Vehicle : TN11TR9999  
Claim Amount : 1350000.0  
  
Highest Claim Vehicle  
TN11TR9999  
1350000.0  
  
Vehicle Found  
Owner : Suresh  
Claim Amount : 60000.0
```